ATIVIDADE DA SEMANA

1. MATERIAL DIDÁTICO:

Esta atividade utilizará arquivos para leitura e escrita de dados, já vistos em atividades anteriores. O novo tópico para esta atividade é busca de dados em matrizes. Para isso, você precisa ter todos os conceitos anteriores bem sedimentados.

1. Material já estudado. Refaça os tutoriais e releia o material caso precise sedimentar os conceitos.
    a. No item Arquivos no Material Didático V. encontra um texto básico sobre arquivos
    b. Leia também: http://www.tutorialspoint.com/python/python_files_io.htm
    c. Refaça no tutorial : http://www.codecademy.com/tracks/python o tópico (V. já fez isso em atividades anteriores)
        i. File Input/Output
2. Material sobre matrizes: leia os seguintes materiais
    a. http://www.linuxtopia.org/online_books/programming_books/python_programming/python_ch20s05.html
    b. http://www.i-programmer.info/programming/python/3942-arrays-in-python.html?start=1
    c. Algoritmos para busca binária em listas: http://rosettacode.org/wiki/Binary_search

Nota importante:

Na forma padrão, listas são copiadas por referência, ou seja, se a e b são listas, a expressão a = b faz com que as variáveis a e b refiram a mesma lista. Para copiar os elementos da lista b em uma lista diferente a, use a = b[:].

2. EXEMPLOS:

2.1. EXEMPLO

(a) Faça uma função que recebe um arquivo com os elementos de uma matriz de números inteiros (cada linha do arquivo corresponde a uma linha da matriz com os números separados por um espaço em branco), e devolve uma matriz com todos os elementos.

(b) Faça um programa que, dado um arquivo com os elementos de uma **matriz quadrada** $A_{n \times n}$ de números inteiros, imprime se a matriz A tem uma linha, coluna ou diagonal principal composta apenas por zeros.

```
import sys
def criaMatriz(nome):
    try:
        arquivo = open(nome,'r')
    except IOError:
        print('Não consegui abrir o arquivo',arquivo)
        sys.exit(0)
    matriz = []
    for linha in arquivo:
        linha_matriz = linha.split()
        matriz.append(linha_matriz)
    arquivo.close()
    return matriz
# função auxiliar para verificar se todos os elementos de uma lista possuem valor zero
def soZeros(lista):
            zeros = True
            for i in lista:
                if int(i) != 0:
                    zeros = False
            return zeros
# Programa Principal
#lê o nome do arquivo como argumento e cria a matriz
nome_arquivo = sys.argv[1]
matriz = criaMatriz(nome_arquivo)
#calcula a ordem da matriz quadrada
tam = len(matriz)
#verifica se tem alguma LINHA da matriz formada só por zeros
linha_zeros = False
i = 0
while not(linha_zeros)  and  i < tam:
            linha_zeros = soZeros(matriz[i])
            i = i + 1
```

```
iif linha_zeros:
        print('Matriz possui linhas nulas')
else:
        print('Matriz NÃO possui linhas nulas')
#verifica se tem alguma COLUNA da matriz é formada só por zeros
coluna_zeros = False
j = 0
while not(coluna_zeros)  and  j < tam:
        coluna_matriz = []
        for i in range(tam):            #constrói uma lista da coluna j da matriz
                coluna_matriz.append(matriz[i][j])
        coluna_zeros = soZeros(coluna_matriz)
        j = j + 1
if coluna_zeros:
        print('Matriz possui colunas nulas')
else:
        print('Matriz NAO possui colunas nulas')
#verifica se a DIAGONAL PRINCIPAL da matriz  é formada só por zeros
diagonal = []
for i in range(tam):            #constrói uma lista com a diagonal principal da matriz
        diagonal.append(matriz[i][i])
if soZeros(diagonal):
        print('Matriz possui diagonal principal nula')
else:
        print('Matriz NAO possui diagonal principal nula')
```

Arquivo input.txt

```
0 0 4
0 0 0
2 2 0
```

Execução do programa: python matriz.lin.col.diagonal.zeros.py input.txt

```
Matriz possui linhas nulas
Matriz NÃO possui colunas nulas
Matriz possui diagonal principal nula
```

Grave o programa acima em um arquivo, crie novos arquivos com dados de matrizes quadradas e teste o programa para novos arquivos.

Acrescente ao programa acima a verificação se a **diagonal secundária** da matriz é formada só por zeros. Teste para as mesmas matrizes que V. criou nos exemplos anteriores.

2.2 EXEMPLO

Um problema comum que temos na prática é precisar encontrar um dado valor em uma lista de valores (já fizemos isso anteriormente). Se precisarmos fazer isso várias vezes em um programa e a lista fornecida possuir milhares de elementos, a tarefa fica muito "cara". Uma estratégia é usar o que é conhecido na literatura por "busca binária" para os casos em que a lista fornecida tem os seus valores ordenados. Por exemplo, se a lista tem seus valores em ordem crescente e quisermos testar se um dado valor **v** pertence à lista, podemos compará-lo com o elemento central da lista. Se ele for maior que o elemento central, já sabemos que só precisamos compará-lo com os elementos da segunda metade da lista. Isso já reduz o esforço computacional de tentar encontrar o valor **v** para apenas a metade dos elementos. Podemos ainda usar a mesma estratégia para essa metade da lista e assim vamos sempre reduzindo o conjunto de valores a serem comparados pela metade até encontrarmos o elemento ou chegarmos a uma lista vazia (quando o elemento não pertence à lista). Veja abaixo uma implementação em python dessa estratégia de busca. Dado um valor e uma lista de valores, a função retorna **True** se o valor pertence à lista, ou **False** caso contrário. Para o programa, dado um valor e uma lista de valores como argumentos, imprimir se o valor pertence ou não à lista.

```
import sys

def buscaBinaria(lista, valor):
        ind_menor = 0                  # índice do primeiro elemento da lista
        ind_maior = len(lista) – 1   # índice do último elemento da lista
        achou = False
        while  ind_menor <= ind_maior  and  not(achou):
                mid = (ind_menor + ind_maior)//2    #índice do element central
                if  lista[mid] > valor:
                        ind_maior = mid - 1
                elif  lista[mid] < valor:
                        ind_menor = mid + 1
                else:
                        achou = True
        return  achou

#Programa Principal

elem = sys.argv[1]
lista_string = sys.argv[2]
lista = lista_string.split(',')

if buscaBinaria(lista,elem):
        print('O elemento pertence à lista')
else:
        print('O elemento NÃO pertence à lista')
```

Execução do programa:

```
python  prog.busca.binaria.py  8  3,4,5,6,8

O elemento pertence à lista
```

Grave o programa acima em um arquivo e teste para outros valores (o que V. quiser).

Modifique a função de busca binária acima para que dado um valor e uma lista de valores, a função retorna **o índice da lista onde o valor foi encontrado** se o valor pertence à lista, ou **-1** se o valor não pertence à lista. Para o programa, dado um valor e uma lista de valores como argumentos, imprimir se o valor pertence ou não à lista. No caso do elemento pertencer à lista, imprimir também o índice da lista (i-ésimo elemento da lista). Se houver elementos repetidos na lista que são iguais ao valor procurado, o índice pode ser qualquer um dos valores repetidos.

3. EXERCÍCIO DE PROGRAMAÇÃO:

O jogo da advinhação consiste em uma pessoa (a advinhadora) tentar advinhar um número

determinado por outra pessoa (o oráculo). A cada passo, a advinhadora sugere um número o oráculo diz se a resposta é maior, menor ou igual ao número sugerido. O jogo termina quando a advinhadora acertao número (isto é, quando o oráculo responde “igual”).
Escreva uma função que advinha um número entre 1 e 100 para o jogo da advinhação.
Uma possível solução é implementar uma busca binária: a cada passo mantemos um intervalo de números possíveis e sugerimos o número no centro do intervalo

Exemplos de execução (suponha que o número certo é 70):

```
python advinha.binario.py
50!
Responda menor,maior ou igual: maior
75!
Responda menor,maior ou igual: menor
62!
Responda menor,maior ou igual: maior
68!
Responda menor,maior ou igual: maior
71!
Responda menor,maior ou igual: menor
69!
Responda menor,maior ou igual: maior
70!
Responda menor,maior ou igual: igual
O numero e' 70
```